Multi-Agent Q-Learning for Competitive Atari Tennis

Jean-Baptiste Ragues[1] and Abdelmoughit Douimia[1]

Université du Littoral Côte d'Opale (ULCO), France
jean.baptiste@etu.univ-littoral.fr, abdelmoughit.douimia@etu.univ-littoral.fr

Abstract. Competitive multi-agent reinforcement learning raises difficulties that are less visible in single-agent settings: the opponent evolves during training, rewards may be sparse, and visual observations must be converted into reliable states before a tabular method can be applied. In this work, we study the Atari 2600 game *Tennis* through the PettingZoo multi-agent interface. We examine the complete processing pipeline from raw frames to object detection, discrete state construction, action selection, reward computation, and Q-table updates. We compare two object-detection strategies, three state representations with 81, 9, and 3 states, and several reward-shaping cues based on score events, ball returns, player–ball proximity, and sprite animation. The experiments reveal a strong dependence on the reliability of the extracted visual data, an asymmetry between the two sides of the court, and a sensitivity to both the delay between decisions and the discretization resolution. The current implementation provides a diagnostic framework for tabular multi-agent Q-learning. A stable end-to-end learning curve remains to be demonstrated.

Keywords: Multi-Agent Reinforcement Learning · Q-Learning · Atari Tennis · PettingZoo · Object Detection · Reward Shaping

1 Introduction

Reinforcement learning (RL) aims to learn a policy that maximizes cumulative reward through interactions with an environment. Q-learning is a classical off-policy value-based algorithm that estimates the quality of taking an action in a given state [1]. In a single-agent setting, the environment is usually treated as stationary. In a multi-agent setting, however, another learning agent may modify its policy over time. From the viewpoint of one learner, the environment is therefore non-stationary [6].

Competitive games provide a natural setting for studying these difficulties. We focus on the Atari 2600 game *Tennis*, available through the Arcade Learning Environment (ALE) [4] and the PettingZoo multi-agent interface [5]. The game involves two opposing players. When one player scores, the other loses the point. The setting can therefore be interpreted as a two-player zero-sum interaction.

The initial objective of the project was to apply tabular multi-agent Q-learning to this visual competitive environment. This raises a practical issue: raw Atari frames cannot be used directly as tabular states because the observation space is extremely large. The images must first be converted into compact and reliable discrete information. The central research question is therefore:

> How can the visual Atari Tennis environment be transformed into meaningful states and rewards that allow tabular multi-agent Q-learning to learn a useful policy?

This report studies the following processing pipeline:

`frame` → `object positions` → `discrete state` → `action` → `reward` → `Q-table update.`

The main contribution is not a claim of solved end-to-end learning. Instead, the report analyzes the data-quality bottlenecks that condition the learning process and identifies the experiments required for a robust final evaluation.

2 Related Work

Q-learning was introduced by Watkins and Dayan as a model-free control algorithm that learns action values from experience [1]. In multi-agent settings, a simple baseline is Independent Q-Learning (IQL), where each agent learns its own value function while treating the other agents as part of the environment [2]. This baseline is attractive because of its simplicity, but it does not explicitly model the non-stationarity created by simultaneously changing policies.

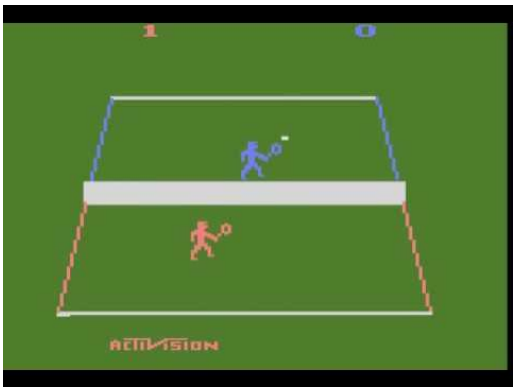


Fig. 1. Atari 2600 Tennis environment. The court is displayed with a perspective effect, while the extracted visual coordinates remain two-dimensional.

Markov games provide a formal framework for multi-agent reinforcement learning in competitive environments [3]. Approaches that model joint actions can represent strategic interactions more explicitly, but they also increase the size of the state–action space. More recent methods often use neural networks to process large observation spaces. In this project, we deliberately begin with a tabular approach in order to study state representation and reward design in a controlled setting.

ALE provides a standard benchmark suite for Atari games [4]. PettingZoo extends this ecosystem with a common multi-agent interface [5], which makes it possible to control both players separately in Atari Tennis.

3 Proposed Methodology

3.1 Environment and Learning Pipeline

Atari Tennis Environment The Atari 2600 game *Tennis* is a two-player competitive environment. Each agent controls one player on its side of the court and attempts to return the ball. The original Atari interface exposes a larger action set, including no-op, fire-related commands, and movement in several directions. For the simplified tabular experiments presented in this report, the decision space is reduced to three horizontal actions: stay in place, move left, and move right.

The native PettingZoo reward is sparse: when an agent scores a point, it receives $+1$; when it loses the point, it receives -1; and during an ongoing rally it receives 0. This reward is faithful to the game score but provides limited immediate feedback about intermediate actions such as positioning or returning the ball.

Tabular Q-Learning For a state s, an action a, a reward r, and a next state s', the Q-table update is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right], \tag{1}$$

where α is the learning rate and γ is the discount factor. During training, actions can be selected with an ε-greedy policy: a random action is selected with probability ε, while an action with maximal estimated value is selected otherwise.

The connection between perception and learning is central. If the estimated object positions are unreliable, the resulting states and rewards may be incorrect. In that case, even a formally correct implementation of Eq. 1 receives noisy updates.

3.2 Object Detection from Atari Frames

Difference Between Consecutive Frames The first approach subtracts two consecutive frames. Because the court background is mostly static, pixels that change between frames are likely to correspond to moving objects. Color thresholds can then help distinguish the two players and the ball.

This approach is simple and computationally inexpensive, but it has important limitations. It requires two distinct frames, it is sensitive to noise, and it may fail to detect objects that remain temporarily immobile. A missing or inaccurate detection directly affects the state provided to the learning algorithm.



(a) Difference between consecutive frames. (b) OpenCV contour and center detection.

Fig. 2. Comparison of the two object-detection approaches.

OpenCV-Based Detection The second approach uses OpenCV to detect object contours and their centers directly in an individual frame. It improves localization and also makes it possible to estimate bounding-box width and height. The centers of the red player, the blue player, and the ball are then used to construct discrete states and reward cues.

3.3 Discrete State Representations

Raw RGB Atari frames contain far too many possible pixel configurations for tabular Q-learning. We therefore tested increasingly simplified state representations.

Global Representation with 81 States The first representation combines the relative position of the ball with respect to both players. For each player, the ball is assigned to one of nine relative regions. A global state is encoded as:

$$s = 9r_{\text{top}} + r_{\text{bottom}}, \qquad r_{\text{top}}, r_{\text{bottom}} \in \{0, \ldots, 8\}, \tag{2}$$

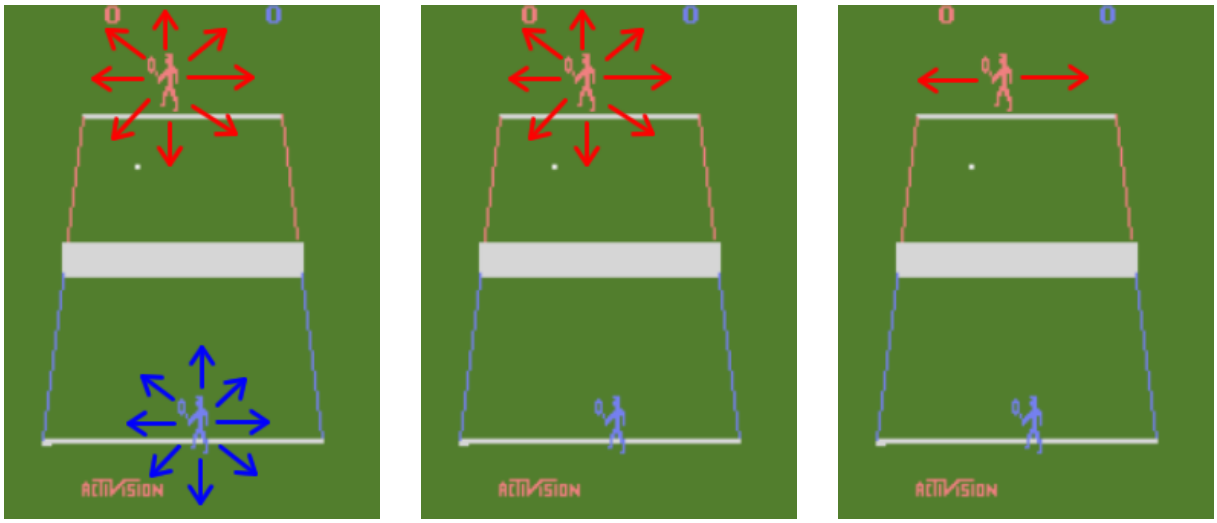
which yields $9 \times 9 = 81$ possible global states. This representation preserves information about both players but requires more exploration.

Individual Representation with 9 States The second representation gives each agent an individual state with nine relative regions around the player: above, above-right, right, below-right, below, below-left, left, above-left, and aligned. This representation is more local and reduces the amount of information processed by each agent.

Simplified Individual Representation with 3 States The third representation keeps only the horizontal relation between the ball and a player:

$$s \in \{\text{ball left},\ \text{ball aligned},\ \text{ball right}\}. \tag{3}$$

The corresponding simplified actions are:



(a) Global 81-state view. (b) Individual 9-state view. (c) Simplified 3-state view.

Fig. 3. Successive simplifications of the state representation.

$$a \in \{\text{stay},\ \text{move left},\ \text{move right}\}. \tag{4}$$

This one-dimensional simplification does not capture the full game, but it is useful for testing whether the reward signal pushes the Q-table toward an expected behavior.

3.4 Reward Design

Native Sparse Score Reward The native PettingZoo reward is based on score events. It is reliable but delayed: many frames may separate a useful positioning action from the final score event. This sparse signal makes credit assignment difficult.

Ball-Return Detection A first shaped reward cue detects a return by storing a short history of ball positions and identifying changes in direction. The closest player can then receive an intermediate positive signal. This feedback is closer in time to a useful action, but it has two main weaknesses:

- at the beginning of an episode, the available position history is insufficient to estimate a reliable direction;
- the apparent perspective of the court can make two-dimensional image-coordinate changes difficult to interpret.

Player–Ball Proximity A second cue detects when the ball is close to, or overlaps with, a player. It is simpler and less sensitive to changes in direction. However, proximity does not guarantee that the ball has actually been returned.

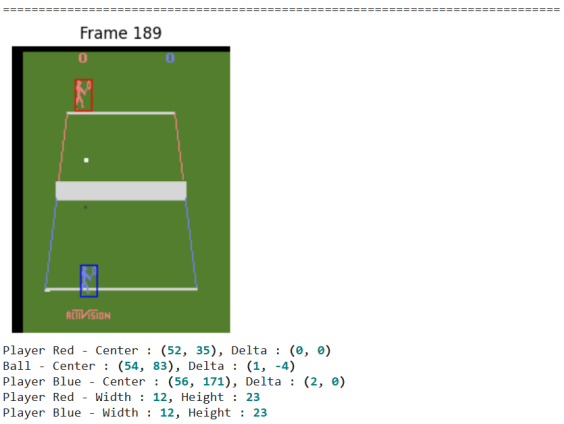
Sprite Animation as an Additional Indicator A third diagnostic cue uses player-sprite animation. In some frames, the detected bounding-box width changes during a movement or return animation. This cue is not sufficient on its own, but it may complement the ball-direction and proximity cues.

The available project archive documents the individual reward cues and the intention to combine them, but it does not record the final coefficients of a combined reward formula. For reproducibility, this report therefore analyzes the cues individually and does not claim a validated weighted combination.

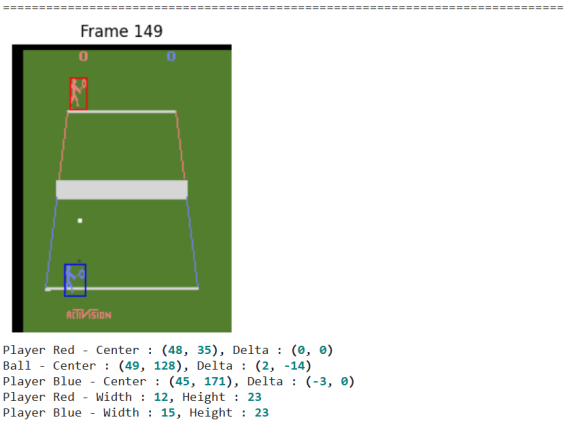
4 Experiments

4.1 Reward-Signal Validation with an Ideal Q-Table

To validate reward signals independently from exploratory action selection, we use the simplified 3-state, 3-action setting. Each agent has a learnable 3×3 Q-table initialized to zero. A second ideal Q-table encodes the expected behavior:



(a) Ball-position diagnostic under perspective.



(b) Sprite-width variation used as an additional cue.

Fig. 4. Visual cues and ambiguities considered during reward design.

Table 1. Expected actions in the simplified setting.

Observed state	Expected action
Ball on the left	Move left
Ball aligned with the player	Stay in place
Ball on the right	Move right

Actions are selected from the ideal table with $\varepsilon = 0$, while updates are applied to the initially empty Q-table. The objective is to test whether a reward signal produces values compatible with the expected policy.

4.2 Preliminary Tests in Game Conditions

A preliminary protocol uses 1000 episodes between the top and bottom players, without switching sides, in order to keep their Q-tables coherent. Unless otherwise stated, the hyperparameters are:

$$\alpha = 0.05, \qquad \gamma = 0.95, \qquad \varepsilon_0 = 0.9, \qquad \text{decay} = 0.9999. \tag{5}$$

The early experiments compare several grid resolutions. The available archive reports the following configurations: 5×10, 10×10, 20×20, 20×30, 30×30, and 20×50 (height $\times$ width). The diagnostic metrics are rally duration in frames and the percentage of actions consistent with the simplified expected behavior.

4.3 Final Sensitivity Tests

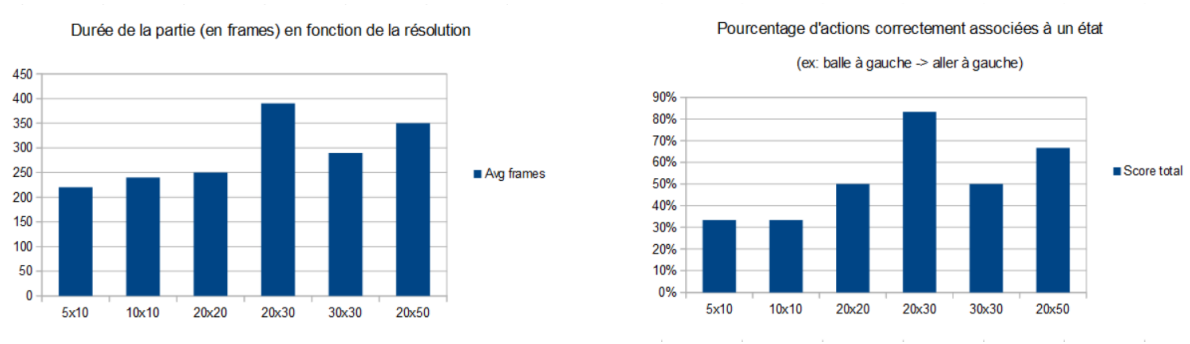
The later experiments focus on perception and decision sensitivity. The delay between decisions is evaluated at 5, 20, 30, 40, and 60 frames. The horizontal grid resolution is evaluated at widths of 10, 15, and 50 cells. These results remain diagnostic: they measure the quality of the perception–decision pipeline rather than the convergence of a complete competitive learning system.

The project archive does not document multiple random seeds or independent repeated runs. The reported plots should therefore be interpreted as preliminary diagnostics rather than statistical estimates with confidence intervals.

5 Results and Discussion

5.1 Preliminary Resolution Study

Figure 5 reports the early comparison of grid resolutions. In this diagnostic setting, the 20×30 grid obtains both the longest rally duration and the largest percentage of actions associated with the expected simplified behavior. The result suggests that an intermediate grid resolution offers a useful compromise: a coarse grid loses information, while a very fine grid can amplify detection errors.



(a) Average rally duration in frames. (b) Correct-action percentage.

Fig. 5. Preliminary sensitivity study for several grid resolutions.

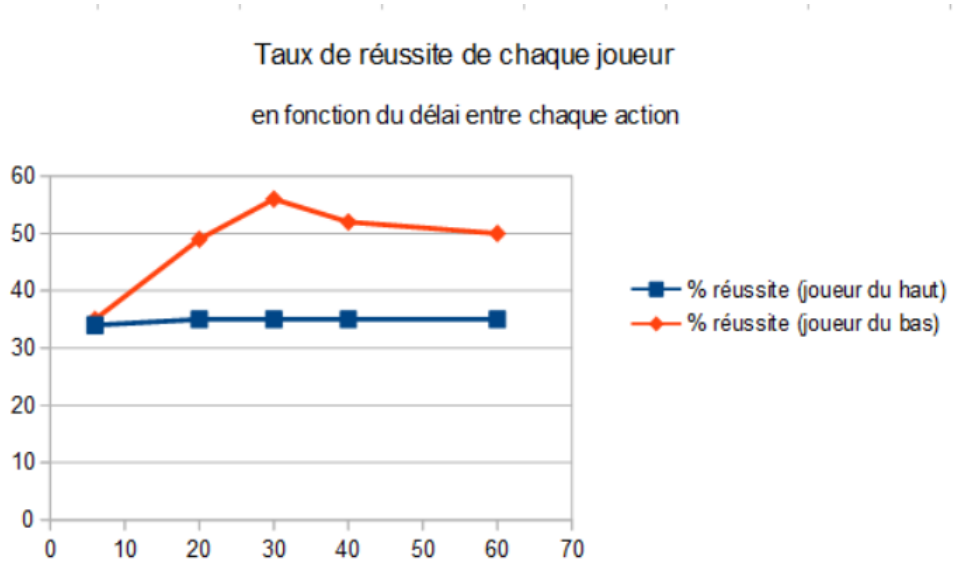


Fig. 6. Percentage of correct actions as a function of the delay between decisions.

5.2 Influence of the Delay Between Decisions

Figure 6 shows the percentage of correct actions as a function of the delay between decisions. The top player remains approximately stable in the mid-thirty-percent range. The bottom player improves when the delay increases up to approximately 30 frames, then decreases slightly for larger values. This suggests a trade-off: acting too frequently can create unstable behavior, while acting too slowly reduces reactivity.

5.3 Influence of Horizontal Resolution

Figure 7 shows the percentage of correct actions for several horizontal grid resolutions. Increasing the width resolution improves the results for the bottom player, while the top player's results remain relatively stable and lower. This asymmetry indicates that the perception and representation pipeline behaves differently depending on the side of the court.

5.4 Interpretation

The experiments support four main observations:

1. a compact state representation is easier to validate than a larger representation;
2. reward shaping can provide more immediate feedback than the native sparse score reward;
3. the usefulness of a reward cue depends strongly on the reliability of the visual data from which it is computed;

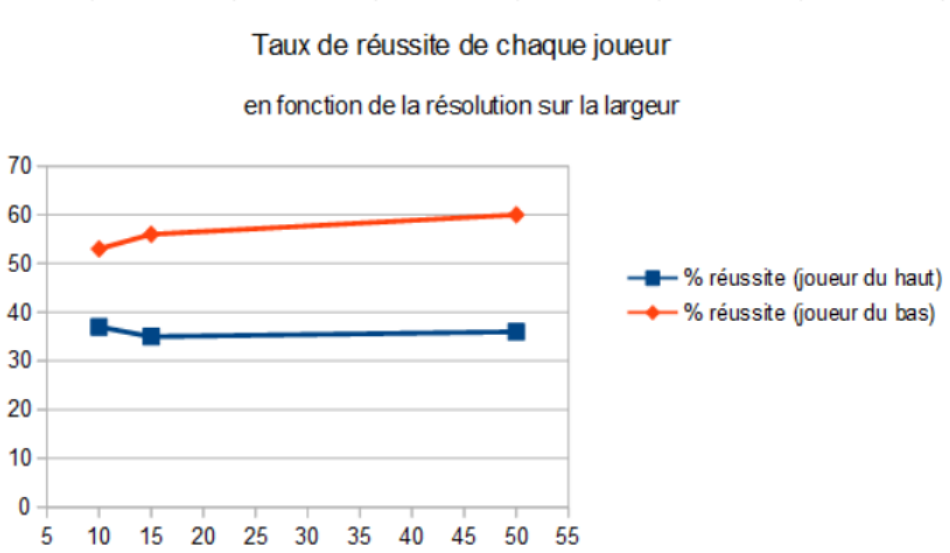


Fig. 7. Percentage of correct actions as a function of the horizontal grid resolution.

4. perception errors can be confused with learning errors.

The last point is especially important. The object-detection component is not separate from the learning problem: it determines the states and rewards used in every Q-table update. A Q-learning implementation can be mathematically correct while still producing poor behavior if its inputs are noisy or delayed.

5.5 Limitations

The current work has several limitations. First, the final coefficients of a combined reward formula were not recorded in the available project archive. Second, the plots measure diagnostic correct-action rates and rally duration, not the convergence of a final competitive learning system. Third, the current experiments do not provide a final plot of episode return, score difference, or learned-policy quality over training episodes with repeated runs and confidence intervals. Finally, the performance difference between the top and bottom players requires further investigation. It may arise from perspective effects, asymmetric detections, or reward-assignment errors.

A complete evaluation should compare at least two perception pipelines and report average episode return, rally duration, score difference, and correct-action rate as functions of training episodes. Multiple random seeds should be used to estimate variability.

6 Conclusion and Future Work

This project studied the application of tabular multi-agent Q-learning to Atari Tennis. The experiments show that the main challenge is not only the Q-learning update itself, but also the construction of reliable states and rewards from visual observations. We compared frame-difference and OpenCV-based object detection, tested three state representations, and explored several intermediate reward cues.

The project provides a clearer understanding of the complete chain from Atari images to learning updates. It also identifies the experiments required to demonstrate stable end-to-end learning. Future work should improve object detection, define and evaluate a reproducible combined reward, compare learning curves across multiple random seeds, and investigate neural-network-based approaches such as Deep Q-Learning when the tabular state representation becomes insufficient.

References

1. Watkins, C.J.C.H., Dayan, P.: Q-learning. Machine Learning **8**(3–4), 279–292 (1992)

2. Tan, M.: Multi-agent reinforcement learning: Independent vs. cooperative agents. In: Proceedings of ICML, pp. 330–337 (1993)
3. Littman, M.L.: Markov games as a framework for multi-agent reinforcement learning. In: Proceedings of ICML, pp. 157–163 (1994)
4. Bellemare, M.G., Naddaf, Y., Veness, J., Bowling, M.: The Arcade Learning Environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research **47**, 253–279 (2013)
5. Terry, J.K., Black, B., Grammel, N., Jayakumar, M., Hari, A., Sullivan, R., Santos, L., Dieffendahl, C., Horsch, C., Perez-Vicente, R., Williams, N., Lokesh, Y., Ravi, P.: PettingZoo: Gym for multi-agent reinforcement learning. In: Advances in Neural Information Processing Systems **34** (2021)
6. Hernandez-Leal, P., Kartal, B., Taylor, M.E.: A survey and critique of multiagent deep reinforcement learning. Autonomous Agents and Multi-Agent Systems **33**, 750–797 (2019)